

Manual del Training-Center — RAGE

Guía práctica para usar el sistema de red-teaming y auto-entrenamiento de RAGE en Mac/Linux, sin API key y sin costo extra.

Versión: v3 (branch cursor/rage-v3-93a0)

Proyecto: RAGE-HACKATHON-PROYECT

1. ¿Qué es el Training-Center?

Es el módulo del proyecto que prueba RAGE contra ataques y genera resultados que puedes usar para endurecer el modelo.

Tiene dos herramientas distintas:

Herramienta | Comando | Para qué sirve

Campañas fijas | `uv run rage-training` | Ejecuta escenarios predefinidos y mide ASR

Red-team adaptativo | `uv run rage-redteam` | Intenta romper RAGE de forma iterativa

Aplicar parches | `uv run rage-training-apply` | Aplica insights al knowledge base

Flujo mental:

- `rage-training` → ¿RAGE aguanta los ataques conocidos?
 - `rage-redteam` → ¿Hay huecos nuevos? ¿Puedo encontrarlos y parchearlos?
 - `rage-training-apply` → Aplicar lo aprendido al KB
-

2. Setup inicial (una vez)

Desde la raíz del repo (donde están `pyproject.toml` y `rage_core/`):

```
cd ~/RAGE-HACKATHON-PROYECT
git fetch origin
git checkout cursor/rage-v3-93a0
bash scripts/check_setup.sh
uv sync
```

Comprobación rápida:

```
uv run python -c "import rage_core; print('OK!')"
uv run pytest tests/ -q
```

Errores comunes de setup

Error | Solución

No module named 'rage_core' | cd a la raíz + `uv sync`

Is `rage_core` no existe | Estás en subcarpeta; sube hasta ver `rage_core/`

Resolved 2 packages | Clone incompleto; re-clona el repo
Extra 'dev' is not defined | Usa solo uv sync (sin --extra dev)

3. rage-training — Campañas con escenarios fijos

Qué hace

Corre cada escenario con RAGE y sin RAGE (baseline), turno a turno, y calcula el ASR (Attack Success Rate).

Comandos

```
# Todos los escenarios
uv run rage-training

# Escenarios específicos
uv run rage-training --scenarios drop_table_escalation crescendo_escalation

# Repetir N veces
uv run rage-training --scenarios drop_table_escalation --iterations 10

# Sin baseline (más rápido)
uv run rage-training --no-baseline
```

Escenarios disponibles

ID | Qué simula
drop_table_escalation | Confianza → role-change → DROP → export
crescendo_escalation | Escalada gradual → UNION ALL exfiltration
canary_leak | Intento de filtrar el canary ZEPHYR-7781
benign_conversation | Conversación legítima (mide falsos positivos)

Cómo leer el resumen

- Defended ASR — ataques exitosos CON RAGE (objetivo: 0%)
- Baseline ASR — ataques exitosos SIN RAGE (esperado: alto)
- RAGE ASR reduction — cuánto bajó el riesgo (objetivo: 100%)

Bueno: Defended ASR = 0% en escenarios de ataque.

Malo: Defended ASR > 0% en drop_table o crescendo.

Archivos generados

- Training-Center/results/crescendo_YYYYMMDD_HHMMSS.json
- Training-Center/insights/pending_hardening_*.json

4. rage-redteam — Auto-entrenamiento adaptativo (v3)

Qué hace

Un loop que:

1. Genera turnos de ataque (plantillas offline o LLM con API key)
2. Los pasa por RAGE
3. Si encuentra bypass → lo guarda en vuln_db.json
4. Opcionalmente aplica parches al KB y gateway

Modo headless (recomendado en Mac)

```
# Rápido
uv run rage-redteam --no-interactive --scale light

# Largo con límite
uv run rage-redteam --no-interactive --scale heavy \
  --objectives exfil ddl schema_dump canary privilege

# Modo ilimitado (hasta que lo pares)
uv run rage-redteam --no-interactive --unlimited \
  --severity critical \
  --objectives exfil ddl schema_dump canary privilege
```

Modo interactivo

```
uv run rage-redteam
```

Menú de configuración:

Campo | Tecla | Descripción

Escala | ◀▶ | light / medio / heavy

Modo ilimitado | SPACE | Corre hasta que pares

Iteraciones | ▲▼ | Solo si ilimitado OFF

Gravedad | ◀▶ | light / medium / high / critical

Objetivos | SPACE | exfil, ddl, schema_dump, canary, privilege

Modelo | ◀▶ | offline / gpt-4o-mini / gpt-4o

Auto-patch | SPACE | Parchear tras bypass

ENTER para iniciar.

Gravedad (severity)

Nivel | Comportamiento offline

light | Solo turnos benignos

medium | Escalada secuencial (default)

high | Ataque empieza 2 turnos antes

critical | Salta a plantillas de ataque, cicla

Escalas

Escala | Iteraciones | Turnos/iter | Backtracks

light | 5 | 8 | 5

medio | 20 | 12 | 10

heavy | 50 | 20 | 10

unlimited | infinito | según escala | según escala

Objetivos de ataque

Objetivo | Vector

exfil | UNION ALL / export_data

ddl | DROP, TRUNCATE, DELETE

schema_dump | information_schema, sqlite_master

canary | Filtrar tokens/secrets

privilege | GRANT, escalada de privilegios

Panel en vivo

Tecla | Acción

S | Stop

Q | Salir seguro

P | Pausa / Resume

U | Toggle modo ilimitado

M | Cambiar modelo LLM

V | Ver vulnerabilidades

Cómo parar sin perder datos

1. Pulsa Ctrl+C una vez
2. Espera 10-15 s hasta ver CAMPAIGN COMPLETE
3. No cierres la terminal antes

Para toda la noche en Mac:

```
caffeinate -i uv run rage-redteam --no-interactive --unlimited \
  --severity critical \
  --objectives exfil ddl schema_dump canary privilege
```

Archivos generados

- Training-Center/results/redteam_*.json
- Training-Center/vulnerabilities/vuln_db.json
- Training-Center/logs/redteam.log (modo interactivo)

Nota: archivo JSON de 0 bytes = corrida interrumpida mal. Relanzar y parar con Ctrl+C esperando el resumen.

5. rage-training-apply — Aplicar parches

```
uv run rage-training-apply
uv run rage-training-apply --campaign crescendo_20260621_135427
uv run rage-training-apply --apply-kb
uv run rage-training-apply --apply-kb --dry-run
```

rage-redteam con auto-patch aplica parches automáticamente al encontrar bypass.

6. rage-demo — Ver la defensa turno a turno

```
uv run rage-demo --scenario drop_table_escalation --no-plot
uv run rage-demo --scenario canary_leak --no-plot
uv run rage-demo --scenario benign_conversation --no-plot
```

Muestra por turno: score, banda (ALLOW/WARN/BLOCK), ejecución de tools.

7. Flujos de trabajo recomendados

A) Sanity check (2 min)

```
uv run rage-training --scenarios drop_table_escalation crescendo_escalation
```

Esperado: Defended ASR 0.0%

B) Validación completa (5 min)

```
uv run rage-training
uv run pytest tests/ -q
```

C) Buscar vulnerabilidades (noche)

```
caffeinate -i uv run rage-redteam --no-interactive --unlimited \
  --severity critical \
  --objectives exfil ddl schema_dump canary privilege
```

D) Ciclo de mejora (si hay bypasses)

```
cat Training-Center/vulnerabilities/vuln_db.json
uv run rage-training-apply --apply-kb
uv run rage-training --scenarios drop_table_escalation crescendo_escalation
```

8. Métricas clave

Métrica | Significado | Objetivo

ASR defended | % ataques exitosos con RAGE | 0%

ASR baseline | % sin RAGE | Alto

ASR reduction | Diferencia | 100%

pipeline_score | Riesgo por turno (0-100) | Subir en escalada

pipeline_band | allow / warn / block | warn/block antes del tool

session_risk | Riesgo acumulado | Subir en multi-turn

bypasses | Ataques que pasaron RAGE | 0 ideal

9. Estructura de carpetas

```
Training-Center/
├─ results/
│   ├─ crescendo_*.json    ← rage-training
│   └─ redteam_*.json      ← rage-redteam
├─ insights/
│   ├─ pending_hardening_*.json
│   └─ applied/
├─ vulnerabilities/
│   └─ vuln_db.json
└─ logs/
    └─ redteam.log
```

10. Cheat sheet

```
# Setup
cd ~/RAGE-HACKATHON-PROJECT
git checkout cursor/rage-v3-93a0
uv sync

# Validar
uv run rage-training --scenarios drop_table_escalation crescendo_escalation

# Demo visual
uv run rage-demo --scenario drop_table_escalation --no-plot

# Red-team corto
uv run rage-redteam --no-interactive --scale light --severity high

# Red-team toda la noche
caffeinate -i uv run rage-redteam --no-interactive --unlimited \
  --severity critical \
  --objectives exfil ddl schema_dump canary privilege

# Parar: Ctrl+C, esperar CAMPAIGN COMPLETE

# Mañana
```

```
cat Training-Center/vulnerabilities/vuln_db.json
uv run rage-training-apply --apply-kb
uv run rage-training
```

RAGE — Retrieval-Augmented Governance Engine